Practical 8: Performance Optimization and Lazy Loading in React

Objective	To improve frontend performance using lazy loading and code splitting techniques.
Architecture / Diagram	
Diagram	Before (single bundle): main.bundle.js ——————————► [Home][Projects][Contact] all loaded upfront After (code-split): main.bundle.js —► loads on first visit Home.chunk.js —► loaded only when / is visited Projects.chunk.js –► loaded only when /projects is visited Contact.chunk.js —► loaded only when /contact is visited
Problem Definition	
Problem Statement	• Optimize the React frontend of the Task Management application by implementing lazy loading for route-based components. • Apply React.lazy() and Suspense to at least the Projects and Contact routes (or equivalent route-based components). • Provide a meaningful fallback UI while a lazy-loaded chunk is loading. • Measure and compare bundle size and load time before and after optimization using browser developer tools. • Document the before/after comparison with screenshots or recorded values.

Steps

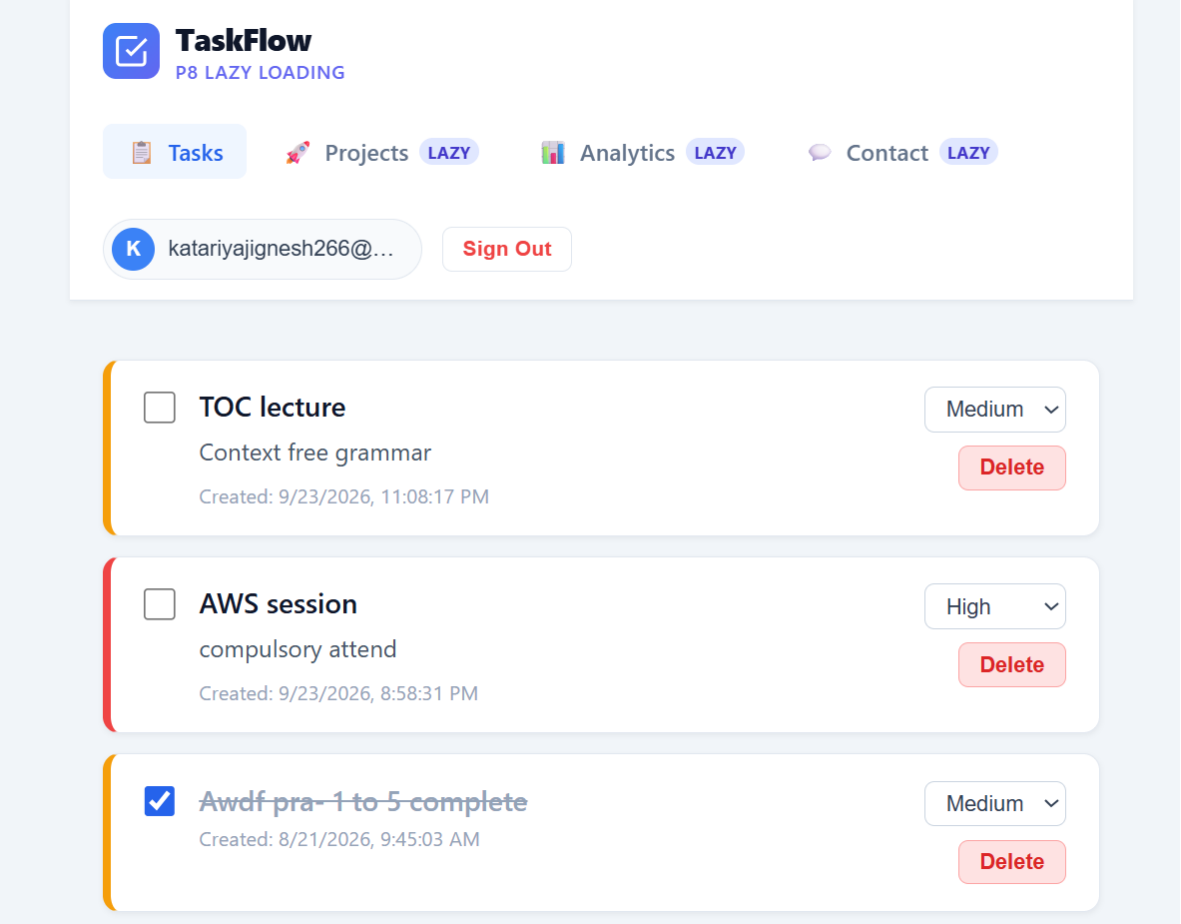
1. Record baseline metrics before optimization build the app and check bundle size:
npm run build
note the output bundle size shown in the terminal
2. Open the Network tab in DevTools, reload the app, and note the total JS transferred and load time.
3. Convert route imports to lazy imports:

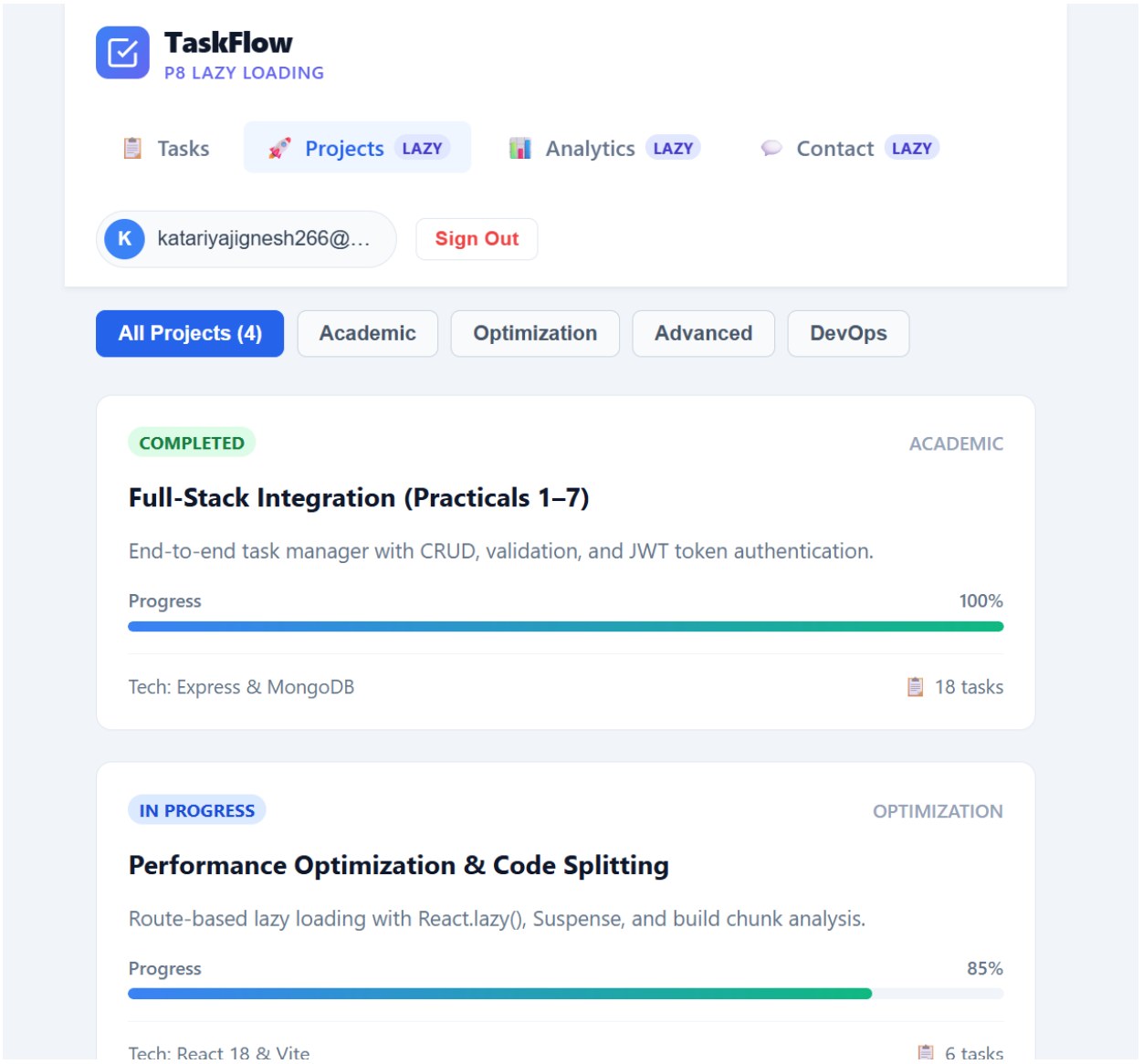
```
import { lazy, Suspense } from 'react';
const Projects = lazy(() => import('./pages/Projects'));
const Contact = lazy(() => import('./pages/Contact'));
```

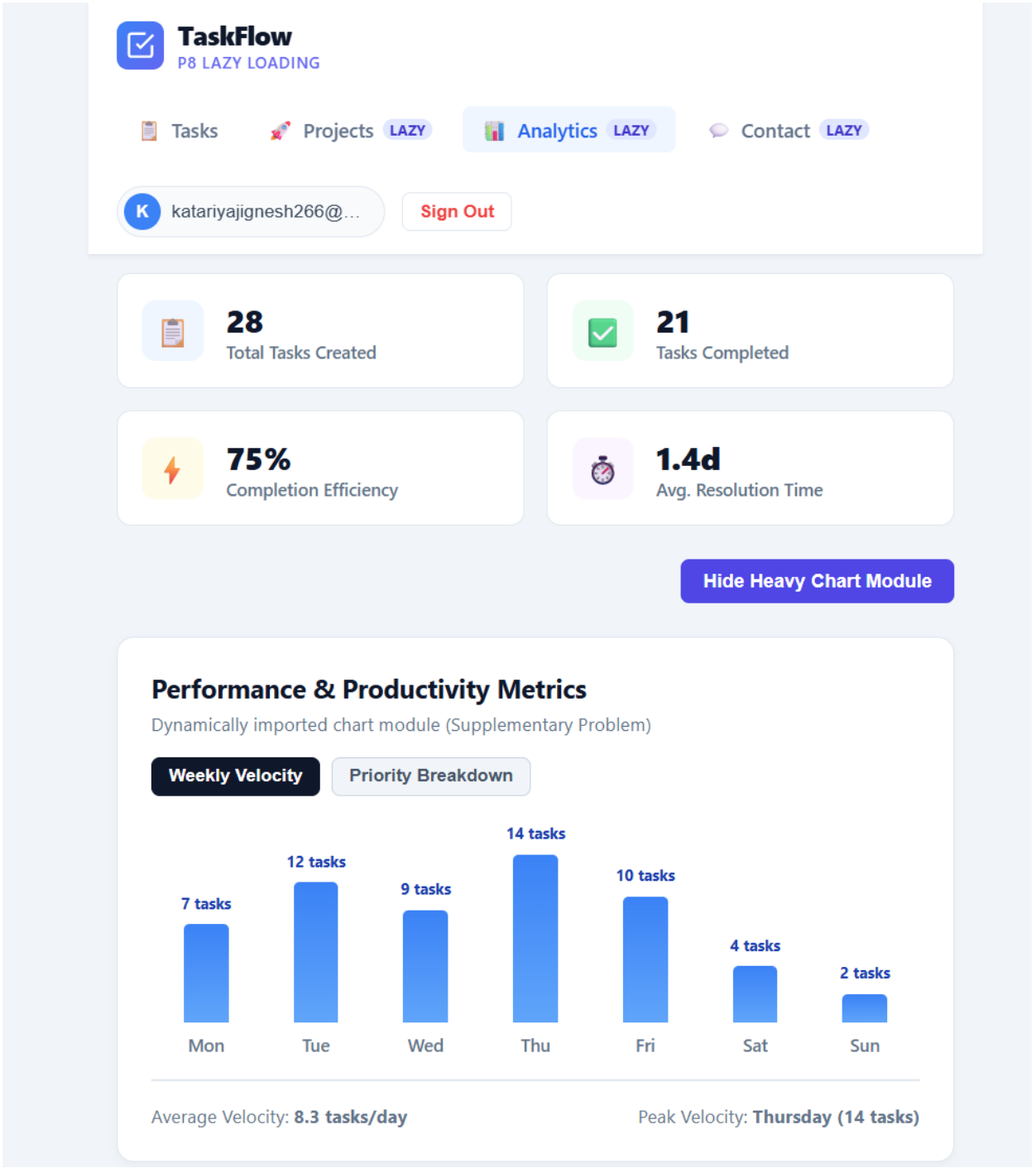
4. Wrap the Routes block with Suspense and a fallback:

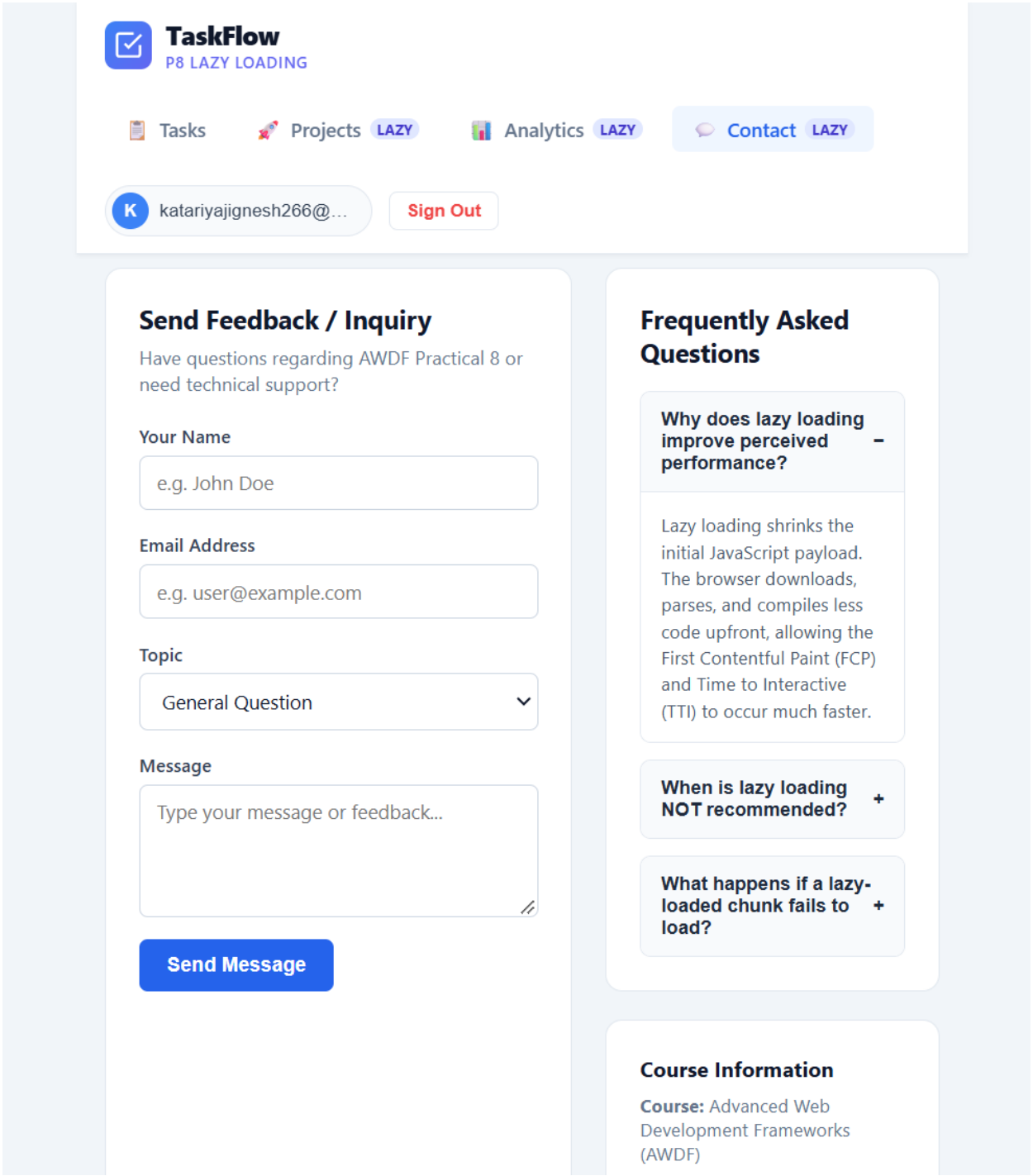
```
<Suspense fallback={<div>Loading page...</div>}>
 <Routes>
 <Route path="/projects" element={<Projects />} />
 <Route path="/contact" element={<Contact />} />
 </Routes>
</Suspense>
```

5. Rebuild the app and confirm separate chunk files are generated for each lazy route.
6. Reload with Network tab open, throttle to "Slow 3G", and observe the fallback UI rendering before the chunk loads.




TaskFlow
P8 LAZY LOADING
Tasks
Projects LAZY
Analytics LAZY
Contact LAZY
katariyajignesh266@...
Sign Out
All Projects (4)
Academic
Optimization
Advanced
DevOps
COMPLETED
ACADEMIC
Full-Stack Integration (Practicals 1–7)
End-to-end task manager with CRUD, validation, and JWT token authentication.
Progress
100%
Tech: Express & MongoDB
18 tasks
IN PROGRESS
OPTIMIZATION
Performance Optimization & Code Splitting
Route-based lazy loading with React.lazy(), Suspense, and build chunk analysis.
Progress
85%
Tech: React 18 & Vite
6 tasks


TaskFlow
P8 LAZY LOADING
Tasks
Projects LAZY
Analytics LAZY
Contact LAZY
K
katariyajignesh266@...
Sign Out
28
Total Tasks Created
21
Tasks Completed
75%
Completion Efficiency
1.4d
Avg. Resolution Time
Hide Heavy Chart Module
Performance & Productivity Metrics
Dynamically imported chart module (Supplementary Problem)
Weekly Velocity
Priority Breakdown
7 tasks
12 tasks
9 tasks
14 tasks
10 tasks
4 tasks
2 tasks
Mon
Tue
Wed
Thu
Fri
Sat
Sun
Average Velocity: 8.3 tasks/day
Peak Velocity: Thursday (14 tasks)


TaskFlow
P8 LAZY LOADING
Tasks
Projects LAZY
Analytics LAZY
Contact LAZY
K
katariyajignesh266@...
Sign Out
Send Feedback / Inquiry
Have questions regarding AWDF Practical 8 or need technical support?
Your Name
e.g. John Doe
Email Address
e.g. user@example.com
Topic
General Question
Message
Type your message or feedback...
Send Message
Frequently Asked Questions
Why does lazy loading improve perceived performance?
Lazy loading shrinks the initial JavaScript payload. The browser downloads, parses, and compiles less code upfront, allowing the First Contentful Paint (FCP) and Time to Interactive (TTI) to occur much faster.
When is lazy loading NOT recommended?
What happens if a lazy-loaded chunk fails to load?
Course Information
Course: Advanced Web Development Frameworks (AWDF)

Key Questions / Analysis	**1. Difference between initial bundle and lazy-loaded chunk?** • **Initial bundle:** Downloaded when the application first loads. • **Lazy-loaded chunk:** Downloaded only when a specific page/component/feature is needed. **2. Why does lazy loading improve perceived performance?** Because the browser downloads **less code initially**, so the app can start and become interactive faster. The remaining code is downloaded later when required. **3. When is lazy loading not worth the complexity?** For a **very small application** with little code, lazy loading may add unnecessary complexity. If the initial bundle is already small, the performance benefit may be too small to justify the extra setup and loading logic.